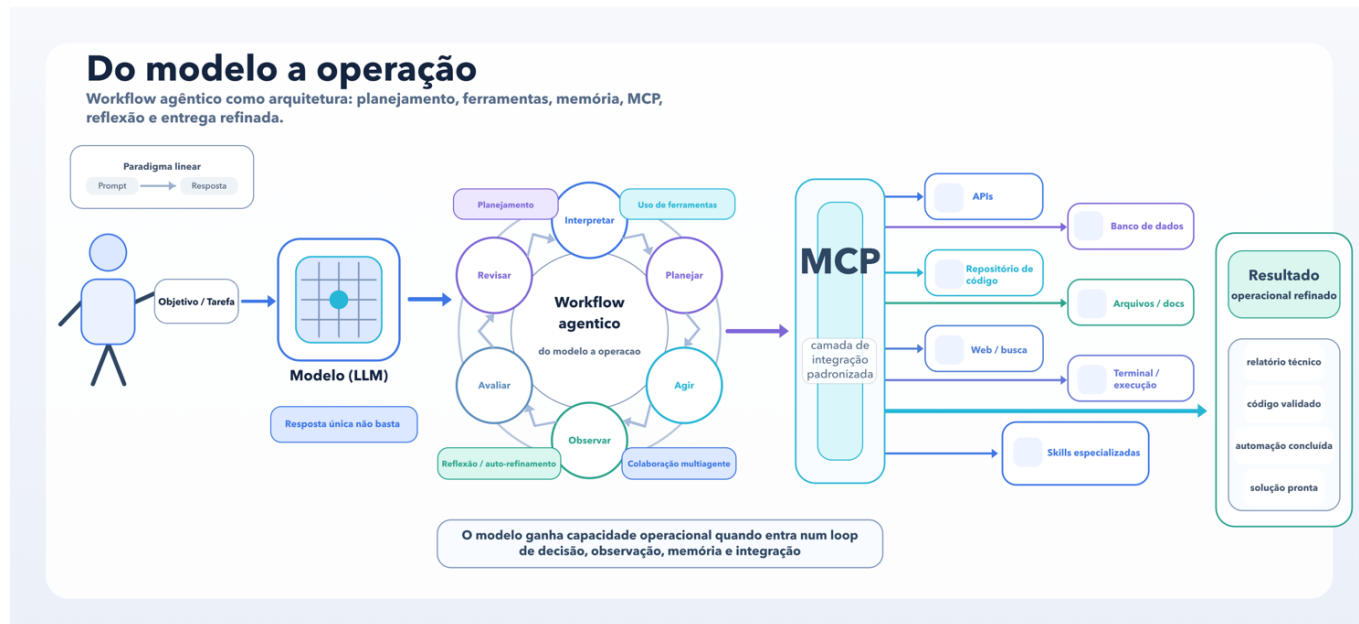


A Arquitetura do Workflow Agêntico

A forma mais comum de interação com modelos de linguagem ainda carrega uma expectativa inadequada: a de que um único prompt bem formulado deva produzir, de imediato, uma resposta correta, completa e pronta para uso. Essa lógica, próxima do chamado zero-shot, trata o modelo como se ele precisasse acertar tudo na primeira tentativa. Em termos práticos, é como exigir que uma pessoa escreva um texto inteiro sem revisar, apagar, reorganizar ou testar ideias ao longo do processo. Em aplicações reais de desenvolvimento de software, automação e integração de serviços, esse pressuposto é frágil. Tarefas úteis raramente dependem apenas de uma resposta inicial; elas exigem interpretação de objetivo, decomposição do problema, uso de recursos externos, validação de resultado e correção de falhas. Por isso, a passagem do modelo à operação marca uma mudança de paradigma: o foco deixa de estar apenas em modelos poderosos e passa a recair sobre sistemas baseados em modelos, nos quais iteração, revisão e controle de fluxo se tornam mais importantes do que a exatidão imediata. Nesse contexto, a velocidade de geração e de reexecução do ciclo pode ser mais valiosa do que a busca obsessiva por uma primeira resposta perfeita, porque permite percorrer rapidamente os laços de análise, correção e melhoria.

Figura 1 - Visão geral da arquitetura do workflow agêntico com articulação entre modelo, workflow, MCP, ferramentas externas e resultado operacional refinado.



Essa mudança ajuda a compreender o que caracteriza um workflow agêntico. Em vez de tratar a IA como um simples mecanismo de geração de texto, o workflow agêntico a insere em uma arquitetura operacional composta por três camadas articuladas. A primeira é o cérebro, isto é, o modelo de linguagem que atua como motor de raciocínio e geração. A segunda é o workflow, entendido como a camada estratégica que organiza a execução, define etapas, mantém estado e controla ciclos de repetição, validação e ajuste. A terceira é a interface, formada por skills, ferramentas, dados e protocolos que permitem ao agente atuar sobre o ambiente externo. Essa divisão é didaticamente útil porque mostra

que a operação não depende apenas da qualidade do modelo, mas da combinação entre raciocínio, lógica de execução e acesso ao mundo externo. Quando essas camadas são tratadas de forma separada, a arquitetura fica mais clara: o modelo pensa, o workflow organiza e a interface conecta. Essa transição pode ser compreendida por meio de uma arquitetura apresentada na Figura 1.

É justamente no workflow que a operação ganha vida. Um agente não se distingue de um chatbot apenas por “saber mais”, mas por operar em um ciclo de interpretação, planejamento, ação, observação, avaliação e revisão. Enquanto um chatbot tende a responder com base no contexto textual disponível, um sistema agêntico pode consultar documentação, recuperar dados, executar comandos, chamar ferramentas, testar uma solução e reformular sua conduta diante do retorno recebido. Essa diferença é estrutural. O valor do sistema deixa de estar em uma resposta isolada e passa a estar em sua capacidade de seguir um processo coerente de trabalho. Em aplicações reais, isso reduz improvisação, aumenta previsibilidade e aproxima a inteligência artificial de práticas consolidadas da engenharia de software, como validação de saídas, tratamento de erro, controle de estado e reexecução orientada por critérios. Em outras palavras, o workflow agêntico transforma geração em operação.

Quatro padrões de orquestração ajudam a explicar como esse workflow funciona na prática. O primeiro é a reflexão e o auto-refinamento. O trabalho SELF-REFINE mostra que um mesmo modelo pode gerar uma resposta inicial, produzir feedback sobre ela e, em seguida, refiná-la iterativamente, sem necessidade de dados adicionais de treinamento. O ponto decisivo não é apenas repetir, mas revisar com base em feedback específico e acionável. Quando o sistema identifica onde falhou e o que precisa mudar, a nova tentativa tende a superar tanto a primeira resposta quanto a simples produção de múltiplas alternativas independentes. O trabalho Reflexion amplia essa lógica ao mostrar que o agente não precisa apenas corrigir uma saída, mas pode aprender com sua própria trajetória. Nessa arquitetura, sinais de falha do ambiente são convertidos em reflexão verbal e armazenados em uma memória episódica, de modo que erros anteriores influenciem decisões futuras. O sistema, assim, deixa de apenas revisar um produto e passa a melhorar seu comportamento entre tentativas, de maneira próxima ao aprendizado humano por tentativa e erro.

O segundo padrão é o uso de ferramentas, ou seja, a capacidade do agente de reconhecer quando seu conhecimento interno não basta e de recorrer a recursos externos para prosseguir. Essas ferramentas podem incluir APIs, motores de busca, compiladores, interpretadores, bancos de dados e arquivos locais. As skills entram justamente nesse ponto, oferecendo especializações e modos organizados de uso desses recursos. O terceiro padrão é o planejamento, que permite decompor tarefas ambíguas em etapas menores, estabelecer uma ordem de execução e recalcular a rota quando uma tentativa intermediária falha. O quarto padrão é a colaboração multiagente, na qual diferentes agentes assumem papéis distintos — como planejador, executor, revisor ou testador — e produzem resultados melhores por meio de divisão de responsabilidade e crítica mútua em fluxo contínuo. Esses padrões não são independentes; eles se reforçam. Um agente planeja melhor quando pode consultar ferramentas, refina melhor quando pode avaliar sua saída e colabora melhor quando há uma arquitetura capaz de manter estado, histórico e coordenação entre papéis.

Para que tudo isso funcione fora do plano conceitual, é necessária uma camada de integração robusta. Historicamente, cada ferramenta usada por um agente exigia uma integração própria, com alto acoplamento e baixa reutilização. Esse modelo gera fragmentação, dificulta manutenção e torna o

sistema pouco escalável. É nesse ponto que o Model Context Protocol (MCP) se torna central. O MCP pode ser entendido como o “encanamento” da operação agêntica: uma interface padronizada que separa a lógica do agente dos servidores que concentram dados, APIs e outros recursos externos. Em vez de cada skill depender de uma implementação isolada, o protocolo organiza o acesso de forma mais uniforme e plugável. Isso não elimina a complexidade do sistema, mas a reposiciona em uma arquitetura mais estável e extensível. Do ponto de vista didático, o modelo continua sendo o cérebro, o workflow continua sendo a lógica de trabalho, e o MCP passa a ser a camada que conecta esse cérebro aos olhos e braços do sistema.

Nessa mesma camada prática, alguns frameworks ajudam a implementar esses princípios. O LangGraph é útil quando se deseja controle fino de estado e de transições em fluxos cíclicos, nos quais o agente precisa voltar a etapas anteriores, testar hipóteses e manter memória da execução. O AutoGen se destaca na orquestração de diálogos entre múltiplos agentes, permitindo estruturar colaboração e revisão distribuída. O CrewAI, por sua vez, enfatiza a delegação por papéis, aproximando a modelagem do funcionamento de equipes humanas que dividem tarefas entre especialistas. Esses frameworks não substituem a arquitetura conceitual; eles a materializam. Em vez de serem o centro da discussão, devem ser compreendidos como instrumentos que tornam operacional aquilo que a teoria já descreve: loops de reflexão, coordenação entre agentes, uso de ferramentas e controle de fluxo.

A consequência mais importante dessa arquitetura talvez esteja na forma como o ser humano passa a interagir com a IA. Em workflows agênticos, a expectativa de resposta imediata perde força. A operação deixa de se parecer com uma pergunta respondida em milissegundos e passa a se aproximar de uma delegação gerencial: o usuário confia uma tarefa ao sistema e aguarda enquanto o agente planeja, consulta recursos, executa etapas, testa resultados, corrige falhas e só então devolve uma solução mais robusta. Essa mudança exige maturidade operacional. Em vez de avaliar apenas a fluidez da resposta inicial, passa-se a valorizar rastreabilidade, consistência, capacidade de revisão e qualidade final do artefato produzido. Para o estudante de Análise e Desenvolvimento de Sistemas, isso é relevante, porque desloca o foco de prompting para arquitetura, processo, integração e controle. Estudar workflow agêntico e MCP, portanto, não é apenas estudar uma tendência recente da IA, mas compreender uma evolução da própria engenharia de sistemas baseados em modelos. Em síntese, a passagem de um modelo que apenas responde para um sistema que realmente opera depende da articulação entre três elementos: um núcleo de raciocínio, um workflow capaz de sustentar iteração e correção, e uma interface padronizada de acesso ao mundo externo. É dessa combinação que emerge a utilidade real dos agentes em aplicações contemporâneas de inteligência artificial.

Referência

MADAAN, Aman et al. Self-Refine: Iterative Refinement with Self-Feedback. [S. l.], 2023. DOI: <https://doi.org/10.48550/arXiv.2303.17651>.

SHINN, Noah et al. Reflexion: Language Agents with Verbal Reinforcement Learning. [S. l.], 2023. DOI: <https://doi.org/10.48550/arXiv.2303.11366>.